

Guía sobre el ciclo **for** en Processing.

Prof. Lescano Carlos

¿Qué es un ciclo for?	1
Estructura del for como conjunto de expresiones	2
El for, los arrays y el uso del índice como expresión	8
Errores comunes	10
for anidados como herramienta para ubicar elementos en pantalla (cuadrículas)	11
for anidados para arrays de 2 dimensiones	12

¿Qué es un ciclo for?

Un ciclo **for** es una estructura de control que permite **repetir un bloque de código una cantidad determinada de veces**. Se utiliza cuando sabemos, o podemos expresar, **cuántas iteraciones queremos realizar**.

En lugar de escribir muchas veces la misma instrucción, el **for** permite automatizar esa repetición usando una **variable de control** (índice). El **for** es fundamental porque permite:

- Recorrer estructuras de datos (como arrays)
- Generar patrones (por ejemplo, en pantalla)
- Automatizar cálculos repetitivos
- Pensar problemas en términos de **procesos repetitivos paso a paso**

Idea básica

Un **for** responde a la lógica: “Repetí esto mientras se cumpla cierta condición, controlando cada paso con una variable.”

Ejemplo simple en Processing

```
for (int i = 0; i < 5; i++) {  
  println(i);  
}
```

Este código imprime:

0
1
2
3
4

¿Qué está pasando?

- Se crea una variable `i`
- `i` empieza en 0
- Mientras `i` sea menor que 5, el código entre {} se repite
- En cada repetición, `i` aumenta en 1

Relación con el pensamiento algorítmico

El `for` obliga a pensar:

- **Dónde empiezo**
- **Hasta cuándo sigo**
- **Cómo avanzo**

Estas tres preguntas son clave para entender no solo el `for`, sino la lógica de muchos algoritmos.

Estructura del `for` como conjunto de expresiones

El `for` no es solo una “estructura que repite”, sino un **conjunto de expresiones que se evalúan en un orden preciso**. Entender esto permite usarlo con mayor control y claridad.

Sintaxis general

```
for (inicialización; condición; actualización) {  
    // cuerpo del ciclo  
}
```

Cada una de las tres partes entre paréntesis es una expresión que cumple un rol específico.

Inicialización (expresión 1)

Se ejecuta **una sola vez**, al comienzo del ciclo.

Ejemplo: `int i = 0;`

- Declara e inicializa la variable de control
- Define el punto de partida

Condición (expresión 2)

Se evalúa **antes de cada iteración**.

Ejemplo: `i < 5`

- Debe ser una expresión booleana (`true` o `false`)
- Mientras sea `true`, el ciclo continúa
- Cuando da `false`, el ciclo termina

Actualización (expresión 3)

Se ejecuta **al final de cada iteración**.

Ejemplo: `i++`

- Modifica la variable de control
- Define cómo avanza el ciclo

Cuerpo del ciclo

Es el bloque de código que se repite:

```
{  
  println(i);  
}
```

- Puede contener una o múltiples instrucciones
- Puede usar la variable de control (`i`); se recomienda no modificarla.
- Puede contener llamadas a funciones y procedimientos.

Ejemplo completo

```
for (int i = 0; i < 5; i++) {  
  println(i);  
}
```

Evaluación paso a paso

El orden real de ejecución es:

1. **Inicialización**
`int i = 0;`
2. **Condición**
`¿i < 5? → true`
3. **Cuerpo**
`println(i);`

4. Actualización

`i++`

5. Vuelve a la condición

`¿i < 5?` → se repite el proceso

Secuencia completa (desplegada)

- `i = 0` → imprime 0 → `i = 1`
- `i = 1` → imprime 1 → `i = 2`
- `i = 2` → imprime 2 → `i = 3`
- `i = 3` → imprime 3 → `i = 4`
- `i = 4` → imprime 4 → `i = 5`
- `i = 5` → condición da `false` → termina

Cada parte del `for` es una **expresión evaluable**, no solo “sintaxis fija”:

- La inicialización define un estado inicial
- La condición decide si seguir o no
- La actualización transforma ese estado

Pensar el `for` de esta manera permite después:

- Ajustar recorridos en arrays
- Diseñar iteraciones más complejas con intención clara

La variable índice y los recorridos básicos

En un `for`, la variable de control, o **índice**, es el elemento que permite organizar la repetición. No es solo un contador: es una **referencia numérica que representa en qué paso del proceso estamos**.

El índice:

- Indica la **iteración actual**
- Permite **controlar cuándo empieza y cuándo termina el ciclo**
- Se usa dentro del cuerpo del `for` para:
 - Acceder a posiciones (por ejemplo, en arrays)
 - Calcular valores
 - Ubicar elementos en pantalla

Ejemplo:

```
for (int i = 0; i < 5; i++) {  
    println("Iteración número: " + i);  
} //Acá i no es solo un número: representa cada paso del proceso.
```

Alcance de la variable

Cuando el índice se declara dentro del `for`:

```
for (int i = 0; i < 5; i++) {  
    println(i);  
}
```

- La variable `i` **solo existe dentro del ciclo**
- No se puede usar fuera de ese bloque

Esto ayuda a:

- Evitar errores
- Mantener el código más ordenado

Buenas prácticas de nombrado

- `i` se usa por convención en recorridos simples; `j`, `k` suelen aparecer en ciclos anidados
- Usar nombres más descriptivos cuando el contexto lo requiere.

Recorridos básicos

El comportamiento del `for` depende directamente de cómo se define el índice. Cambiar la inicialización, la condición o la actualización modifica el recorrido.

Conteo ascendente

Es el más común:

```
for (int i = 0; i < 5; i++) {  
    println(i);  
}
```

- Empieza en 0
- Avanza de a 1
- Termina antes de 5

Resultado: 0, 1, 2, 3, 4

Conteo descendente

Se invierte la lógica:

```
for (int i = 4; i >= 0; i--) {  
  println(i);  
}
```

- Empieza en un valor alto
- Disminuye de a 1
- Termina en 0

Resultado: 4, 3, 2, 1, 0

Cambios en el paso (incrementos distintos de 1)

El índice no es solo un contador automático: es una **variable que se diseña** según el problema; El índice no tiene que avanzar siempre de a 1. Se puede modificar el “ritmo” del recorrido.

De a 2:

```
for (int i = 0; i < 10; i += 2) {  
  println(i);  
}
```

Resultado: 0, 2, 4, 6, 8

De a 5:

```
for (int i = 0; i < 20; i += 5) {  
  println(i);  
}
```

Resultado: 0, 5, 10, 15

Modificar:

- **Dónde empieza** → cambia el origen
- **Hasta dónde llega** → cambia el límite
- **Cómo avanza** → cambia la forma del recorrido

Cambio en el paso aplicado a la pantalla (distribución equidistante)

Hasta ahora el índice avanzaba de a 1, 2, 5, etc. Pero en Processing aparece un uso muy importante: usar el **for** para **recorrer posiciones en la pantalla con un paso calculado**.

Ejemplo: Queremos dibujar una cierta cantidad de objetos **equidistantes** a lo largo del eje X. Para eso:

1. Definimos cuántos objetos queremos
2. Calculamos un **separador**
3. Usamos ese separador como incremento del índice

```
int cantidad = 10;
```

```
float separador = width / cantidad;
```

```
for (float x = 0; x < width; x += separador) {  
    ellipse(x, height/2, 20, 20);  
}
```

¿Qué está pasando?

- **cantidad** define cuántos objetos queremos
- **separador** divide el ancho de la pantalla en partes iguales
- El índice ya no es **i**, sino **x**, porque representa una **posición**
- En cada iteración:
 - **x** avanza exactamente una “sección” de la pantalla

Resultado conceptual

Se dibujan círculos:

- Desde **x = 0**
- Hasta **x < width**
- Separados siempre por la misma distancia

Relación con el cambio de paso

Antes: **i++**

Ahora: **x += separador**

El “ritmo” del ciclo ya no es fijo (1, 2, 5), sino que depende de una **expresión**:
width / cantidad.

El for, los arrays y el uso del índice como expresión

Cuando se trabaja con arrays, el **for** se vuelve una herramienta central porque permite **recorrer todas sus posiciones de manera controlada**. En este contexto, el índice deja de ser solo un contador y pasa a representar directamente una **posición dentro de la estructura**.

Relación entre índice y posición

En un array de una dimensión: `int[] valores = new int[5];`

Las posiciones válidas son: 0, 1, 2, 3, 4

El índice del `for` coincide exactamente con esas posiciones:

```
for (int i = 0; i < 5; i++) {  
    println(valores[i]);  
}
```

- `i` → índice del ciclo
- `valores[i]` → acceso a la posición `i` del array

Recorrido completo de un array: dos estrategias

Al recorrer un array con un `for`, es importante definir correctamente el **límite del recorrido**. Existen dos estrategias válidas según cómo se esté pensando el problema.

Estrategia 1: usar `length` (tamaño propio del array)

En lugar de escribir el tamaño “a mano”, se usa la propiedad del array: `valores.length`

Ejemplo:

```
for (int i = 0; i < valores.length; i++) {  
    println(valores[i]);  
}
```

Ventajas:

- El recorrido se adapta automáticamente al tamaño del array
- Evita errores si cambia la cantidad de elementos
- Es más robusto cuando el array ya existe o viene de otro proceso

Estrategia 2: usar una variable `cantidad`

Se define una variable que controla tanto el tamaño del array como el recorrido:

```
int cantidad = 5;  
int[] valores = new int[cantidad];  
for (int i = 0; i < cantidad; i++) {  
    println(valores[i]);  
}
```

Ventajas:

- Hace explícita la intención (cuántos elementos se quieren trabajar)
- Permite reutilizar esa cantidad en otras partes del programa
- Es útil cuando el tamaño forma parte del problema (por ejemplo, cantidad de objetos en pantalla)

Relación entre ambas estrategias

En este caso: `valores.length == cantidad`

Pero conceptualmente:

- `length` → propiedad del array
- `cantidad` → decisión del programa

Ambas estrategias son correctas, pero responden a enfoques distintos:

- Usar `length` → cuando el array ya define su tamaño
- Usar `cantidad` → cuando el tamaño es una variable del problema

Elegir una u otra depende de qué se quiere destacar en el código: la estructura de datos o la lógica del programa. En muchos ejercicios yo les voy a pedir que ejecuten un programa que se adapte a un número variable de elementos. Ahí es necesario que usen **cantidad**.

Acceso y modificación de elementos

Lectura: `println(valores[i]);`

Escritura: `valores[i] = i * 10;`

Ejemplo:

```
for (int i = 0; i < valores.length; i++) {  
    valores[i] = i * 10;  
}
```

Resultado conceptual: `[0, 10, 20, 30, 40]`

Expresiones dentro del `for` aplicadas a arrays

El índice se puede usar como parte de **cálculos más complejos**.

Generación de valores

```
for (int i = 0; i < valores.length; i++) {  
    valores[i] = i * i;  
}
```

Resultado:

`[0, 1, 4, 9, 16]`

Dependencia entre posición y contenido

El valor depende directamente de la posición: `valores[i] = i * 5;`

Pero también se pueden usar relaciones más complejas: `valores[i] = valores[i-1] + 2;`

(esto requiere cuidado con el caso inicial, ya que si bien existe la posición 0, la posición 0-1 no existe) lo que lleva a

Errores comunes

Índices fuera de rango

Acceder a posiciones inexistentes : `valores[-1];` // inválido
`valores[5];` // inválido si `length = 6`

```
for (int i = 0; i <= valores.length; i++) {  
  println(valores[i]); // ERROR}
```

Problema:

- `valores.length` no es una posición válida
- El último índice siempre es `length - 1`

Correcto:

```
for (int i = 0; i < valores.length; i++) {  
  println(valores[i]);  
}
```

Dependencias mal definidas

`valores[i] = valores[i-1] + 2;`

Si `i = 0`, esto falla. Se debe ajustar:

`valores[0] = 0;`

```
for (int i = 1; i < valores.length; i++) {  
  valores[i] = valores[i-1] + 2;  
}
```

for anidados como herramienta para ubicar elementos en pantalla (cuadrículas)

Los `for` anidados permiten combinar dos recorridos: uno dentro de otro. En Processing esto se usa directamente para trabajar en dos dimensiones, por ejemplo, **recorrer filas y columnas de la pantalla**.

- Un `for` controla el eje X (columnas)

- Otro **for** controla el eje Y (filas)

Cada combinación de ambos índices define una **posición en pantalla**.

Ejemplo: grilla

```
int columnas = 10;
int filas = 6;

void setup() {
  float sepX = width / columnas;
  float sepY = height / filas;

  for (int i = 0; i < columnas; i++) {
    for (int j = 0; j < filas; j++) {
      float x = i * sepX;
      float y = j * sepY;
      ellipse(x, y, 10, 10);
    }
  }
}
```

- **i** recorre las columnas (horizontal)
- **j** recorre las filas (vertical)
- Cada par (**i, j**) genera una posición (**x, y**)

Esto produce una **cuadrícula de puntos distribuidos uniformemente**.

Forma de leer un **for** anidado

Para cada valor de **i**: → se recorre **completamente** el ciclo de **j**

Es decir:

- **i = 0** → se dibuja toda una columna de puntos
- **i = 1** → se dibuja la siguiente columna
- etc.

for anidados para arrays de 2 dimensiones

Cuando se trabaja con arrays de 2D, los **for** anidados permiten **recorrer todas las posiciones de la estructura**, de manera similar a como se recorre una cuadrícula en pantalla. Un array 2D se puede pensar como una tabla:

- Filas

- Columnas

Y cada elemento se accede con dos índices: `array[columna][fila]`

Declaración de un array 2D

```
int filas = 4;
int columnas = 5;

int[][] datos = new int[columnas][filas];
```

Recorrido completo con `for` anidados

```
for (int i = 0; i < columnas; i++) {
    for (int j = 0; j < filas; j++) {
        println(datos[i][j]);
    }
}
```

¿Qué representa cada índice?

- `i` → columnas
- `j` → fila

Cada combinación (`i`, `j`) accede a una celda del array.

Generación de valores

Se pueden llenar los datos usando el índice como parte de una expresión:

```
for (int i = 0; i < filas; i++) {
    for (int j = 0; j < columnas; j++) {
        datos[i][j] = i + j;
    }
}
```

Resultado conceptual:

```
0 1 2 3 4
1 2 3 4 5
2 3 4 5 6
3 4 5 6 7
```

Dependencia entre posición y contenido

También se pueden generar patrones más claros: `datos[i][j] = i * 10 + j;`

Resultado:

```
00 01 02 03 04
10 11 12 13 14
20 21 22 23 24
30 31 32 33 34
```

Esto permite “ver” la posición expresada dentro del valor. estos datos luego pueden representar el atributo de algún elemento visual.

Uso de `length` en 2D

En arrays 2D:

- `datos.length` → cantidad de filas
- `datos[i].length` → cantidad de columnas de esa fila

Ejemplo:

```
for (int i = 0; i < datos.length; i++) {
    for (int j = 0; j < datos[i].length; j++) {
        println(datos[i][j]);
    }
}
```

El `for` anidado en arrays 2D replica exactamente la lógica vista en pantalla:

- Un índice recorre filas
- Otro recorre columnas
- Cada par `(i, j)` representa una posición

Pero en este caso, en lugar de dibujar se accede, modifica o genera información dentro de una estructura de datos.